**ASSESSMENT TOOL 1-DATABASE DESIGN AND CONNECTIVITY**

| | |
|---|---|
| **Course Code: DSA0504** | **Course Title: Query Processing for Data Science** |
| **CO Assessed: CO2** | **Assessment: Tool 1 – Database design and connectivity** |
| **Weightage: 40** | **Total Marks: 25 Marks** |
| **CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)** | |
| **Student Name:J.Subhaashree** | **Reg. No.:192424008** |
| **Section / Batch:2024** | **Date of Submission:3/8/26** |
| **Faculty In-charge: Dr. R.Mahesh Muthulakshmi** | **Project Mode: Individual Submission** |

# 1) Online Course Registration System

## Objective

- To design an SQLite database for maintaining student and course registration details.
- To establish database connectivity using Python and SQLite.
- To create Student and Course_Registration tables with primary and foreign keys.
- To insert sample records into the tables.
- To retrieve the details of students and their registered courses using an SQL JOIN operation.

## CODE

online course.py - C:/Users/subha/OneDrive/Desktop/Query processing/online course.py (3.12.10)

File Edit Format Run Options Window Help

```
import sqlite3

# Connect to database
conn = sqlite3.connect("college.db")

# Create cursor
cur = conn.cursor()

# Create Student table
cur.execute("""
CREATE TABLE IF NOT EXISTS Student(
    student_id INTEGER PRIMARY KEY,
    student_name TEXT NOT NULL
)
""")

# Create Course_Registration table
cur.execute("""
CREATE TABLE IF NOT EXISTS Course_Registration(
    reg_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    course_name TEXT NOT NULL,
    FOREIGN KEY(student_id) REFERENCES Student(student_id)
)
""")

# Insert data into Student table
cur.execute("INSERT INTO Student VALUES (1, 'Arun')")
cur.execute("INSERT INTO Student VALUES (2, 'Priya')")
cur.execute("INSERT INTO Student VALUES (3, 'Kavin')")

# Insert data into Course_Registration table
cur.execute("INSERT INTO Course_Registration VALUES (101, 1, 'Python')")
cur.execute("INSERT INTO Course_Registration VALUES (102, 2, 'Java')")
cur.execute("INSERT INTO Course_Registration VALUES (103, 3, 'DBMS')")

# Join query
cur.execute("""
SELECT Student.student_name,
       Course_Registration.course_name
FROM Student
INNER JOIN Course_Registration
ON Student.student_id = Course_Registration.student_id
""")

# Display output
result = cur.fetchall()
```

**OUTPUT**

```
Student Name     Course
Arun              Python
Priya             Java
Kavin             DBMS
```

## Description

The Online Course Registration System is designed to maintain information about students and the courses they have registered for using an SQLite database. The system consists of two related tables: Student and Course_Registration. The Student table stores details such as student ID and student name, while the Course_Registration table stores course registration details.

Database connectivity is established using Python's `sqlite3` module. The `connect()` function creates a connection between the Python program and the SQLite database, and the `cursor()` method is used to execute SQL queries. The program creates the required tables, inserts sample records, and retrieves the details of students along with their registered courses using an SQL JOIN operation.

Finally, the changes are saved using the `commit()` method, and the database connection is closed using the `close()` method.

## Database Connectivity Steps

1. Import the `sqlite3` module.
2. Establish a connection using `sqlite3.connect()`.
3. Create a cursor object using `cursor()`.
4. Create the required tables.
5. Insert student and course records.
6. Retrieve data using the JOIN query.
7. Save the changes using `commit()`.
8. Close the connection using `close()`.

## Conclusion

The Online Course Registration System was successfully implemented using Python and SQLite. The database connection was established, the required tables were created, sample records were inserted, and the student details along with their registered courses were retrieved using an SQL JOIN query.

# 2) Banking Account Management System

## Objective

- To design an SQLite database for storing customer account information.
- To establish database connectivity using Python and SQLite.
- To create an Account table with appropriate attributes and constraints.
- To insert customer account details into the database.
- To update the account balance of a specified customer and display all records.

## CODE

banking.py - C:/Users/subha/OneDrive/Desktop/Query processing/banking.py (3.12.10)

File Edit Format Run Options Window Help

```python
import sqlite3

# Connect to database
conn = sqlite3.connect("bank.db")

# Create cursor
cur = conn.cursor()

# Create Account table
cur.execute("""
CREATE TABLE IF NOT EXISTS Account(
    account_no INTEGER PRIMARY KEY,
    customer_name TEXT NOT NULL,
    account_type TEXT,
    balance REAL
)
""")

# Insert sample records
cur.execute("INSERT INTO Account VALUES (1001, 'Arun', 'Savings', 25000)")
cur.execute("INSERT INTO Account VALUES (1002, 'Priya', 'Current', 30000)")
cur.execute("INSERT INTO Account VALUES (1003, 'Kavin', 'Savings', 45000)")
cur.execute("INSERT INTO Account VALUES (1004, 'Divya', 'Current', 50000)")
cur.execute("INSERT INTO Account VALUES (1005, 'Rahul', 'Savings', 35000)")

# Update account balance
cur.execute("""
UPDATE Account
SET balance = 40000
WHERE customer_name = 'Priya'
""")

# Retrieve all records
cur.execute("SELECT * FROM Account")

records = cur.fetchall()

print("Account No\tCustomer Name\tAccount Type\tBalance")

for row in records:
    print(row[0], "\t", row[1], "\t\t", row[2], "\t\t", row[3])

# Save changes
conn.commit()

# Close connection
conn.close()
```

**OUTPUT**

```
Account No      Customer Name    Account Type     Balance
1001      Arun           Savings              25000.0
1002      Priya          Current              40000.0
1003      Kavin          Savings              45000.0
1004      Divya          Current              50000.0
1005      Rahul          Savings              35000.0
```

## Description

The Banking Account Management System is designed to maintain customer account information using an SQLite database. The system contains an Account table with attributes such as Account Number, Customer Name, Account Type, and Balance.Database connectivity is established using Python's `sqlite3` module. The `connect()` function creates a connection between the Python program and the SQLite database. A cursor object is created using the `cursor()` method to execute SQL commands.The program creates the Account table, inserts customer account details, updates the balance of a specified customer, and displays all customer records stored in the database.Finally, the changes are saved using the `commit()` method, and the database connection is closed using the `close()` method. The question asks for these operations.

## Database Connectivity Steps

1. Import the `sqlite3` module.
2. Connect to the database using `sqlite3.connect()`.
3. Create a cursor object.
4. Create the Account table.
5. Insert customer records.
6. Update the account balance.
7. Display all records.
8. Save the changes using `commit()`.
9. Close the database connection.

### Conclusion

The Banking Account Management System was successfully implemented using Python and SQLite. The database connection was established, the Account table was created, customer details were inserted, account balances were updated, and all records were displayed successfully.

# 3) Inventory Management System

## Objective

- To design an SQLite database for maintaining product inventory details.
- To establish database connectivity using Python and SQLite.
- To create a Product table with suitable attributes.
- To insert sample product records into the database.
- To identify products with low stock, update their quantities, and display the updated inventory.

## CODE

inventory.py - C:/Users/subha/OneDrive/Desktop/Query processing/inventory.py (3.12.10)

File Edit Format Run Options Window Help

```
import sqlite3

# Connect to database
conn = sqlite3.connect("inventory.db")

# Create cursor
cur = conn.cursor()

# Create Product table
cur.execute("""
CREATE TABLE IF NOT EXISTS Product(
    product_id INTEGER PRIMARY KEY,
    product_name TEXT NOT NULL,
    category TEXT,
    quantity INTEGER,
    unit_price REAL
)
""")

# Insert sample records
cur.execute("INSERT INTO Product VALUES (101, 'Laptop', 'Electronics', 5, 50000)")
cur.execute("INSERT INTO Product VALUES (102, 'Mouse', 'Electronics', 15, 500)")
cur.execute("INSERT INTO Product VALUES (103, 'Keyboard', 'Electronics', 8, 1000)")
cur.execute("INSERT INTO Product VALUES (104, 'Notebook', 'Stationery', 20, 50)")
cur.execute("INSERT INTO Product VALUES (105, 'Pen', 'Stationery', 7, 20)")

# Display products with quantity less than 10
print("Products with quantity less than 10:")

cur.execute("SELECT * FROM Product WHERE quantity < 10")

rows = cur.fetchall()

for row in rows:
    print(row)

# Update stock quantity
cur.execute("""
UPDATE Product
SET quantity = quantity + 10
WHERE quantity < 10
""")

# Display updated inventory
print("\nUpdated Inventory:")

cur.execute("SELECT * FROM Product")
```

**OUTPUT**

```
Products with quantity less than 10:
(101, 'Laptop', 'Electronics', 5, 50000.0)
(103, 'Keyboard', 'Electronics', 8, 1000.0)
(105, 'Pen', 'Stationery', 7, 20.0)

Updated Inventory:
ID      Name            Category        Quantity        Price
101      Laptop          Electronics     15              50000.0
102      Mouse   Electronics     15              500.0
103      Keyboard        Electronics     18              1000.0
104      Notebook        Stationery      20              50.0
105      Pen     Stationery      17              20.0
```

# Description

The Inventory Management System is designed to maintain product inventory information using an SQLite database. The system contains a Product table with attributes such as Product ID, Product Name, Category, Quantity, and Unit Price.

Database connectivity is established using Python's `sqlite3` module. The `connect()` function creates a connection between the Python program and the SQLite database, and the `cursor()` method is used to execute SQL commands.

The program creates the Product table, inserts sample product records, identifies products with quantity less than 10, updates their stock quantity, and displays the updated inventory details.Finally, the changes are saved using the `commit()` method, and the database connection is closed using the `close()` method.

## Database Connectivity Steps

1. Import the `sqlite3` module.
2. Establish a connection using `sqlite3.connect()`.
3. Create a cursor object using `cursor()`.
4. Create the Product table.
5. Insert sample product records.
6. Identify products with quantity less than 10.
7. Update the stock quantity.
8. Display the updated inventory.
9. Save the changes using `commit()`.
10. Close the connection.

## Conclusion

The Inventory Management System was successfully implemented using Python and SQLite. The database connection was established, the Product table was created, sample product records were inserted, products with low stock were identified and updated, and the updated inventory details were displayed successfully.